UNIVERSIDAD AUTÓNOMA DE SINALOA

Facultad de Informática Culiacán
Licenciatura en Informática

Análisis y Diseño Orientado a Objetos

Fundamentos de Programación

M.C. Gerardo Beltrán

Programación Orientada a Objetos

Pilares de la Orientación a Objetos

¿Qué es una clase?

- Para el filósofo…
  - Un artefacto de *clas*ificación humana
  - *Clas*ificamos según un comportamiento o atributos comunes
  - Acordamos descripciones y nombres de *clases* útiles
  - Creamos vocabulario; nos comunicamos; ¡pensamos!

- Para el programador orientado a objetos…
  - Una construcción sintáctica con nombre que describe un comportamiento y atributos comunes
  - Una estructura de datos que incluye datos y funciones

¿Qué es un objeto?

- Un objeto es una instancia de una clase

- Los objetos se caracterizan por:
  - Identidad: Los objetos se distinguen unos de otros
  - Comportamiento: Los objetos pueden realizar tareas
  - Estado: Los objetos contienen información

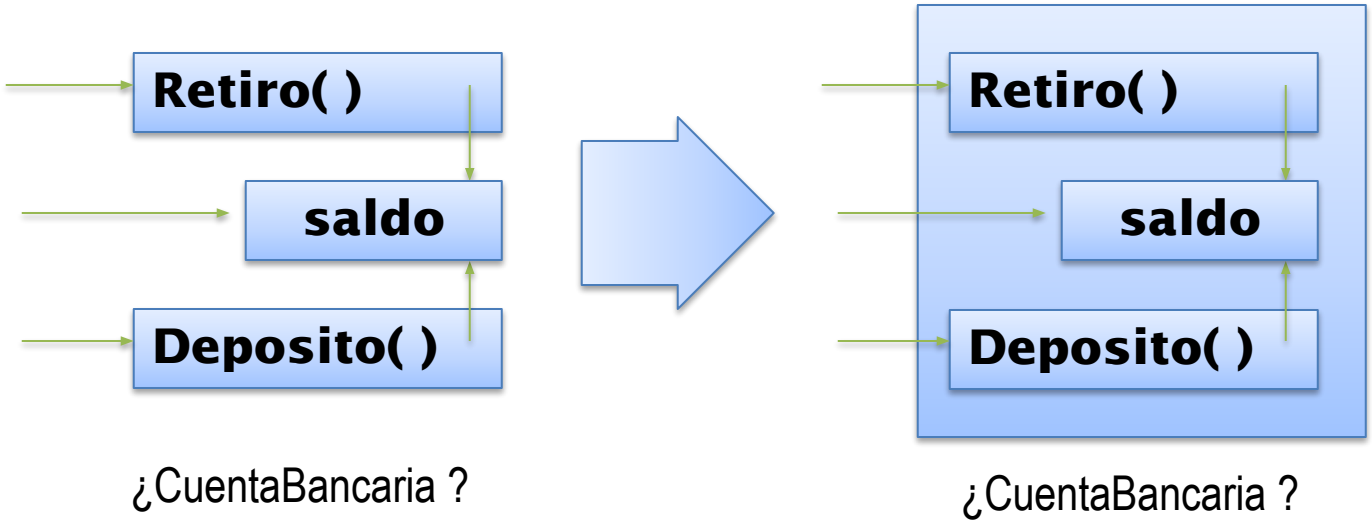
Abstracción

- La abstracción es ignorancia selectiva
  - Decidir qué es importante y qué no lo es
  - Concentrarse en lo importante y depender de ello
  - Ignorar lo que no es importante y no depender de ello
  - Usar encapsulación para forzar una abstracción

El objetivo de la abstracción es no perderse en vaguedades y crear un nuevo nivel semántico en el que se pueda ser absolutamente preciso.
Edsger Dijkstra

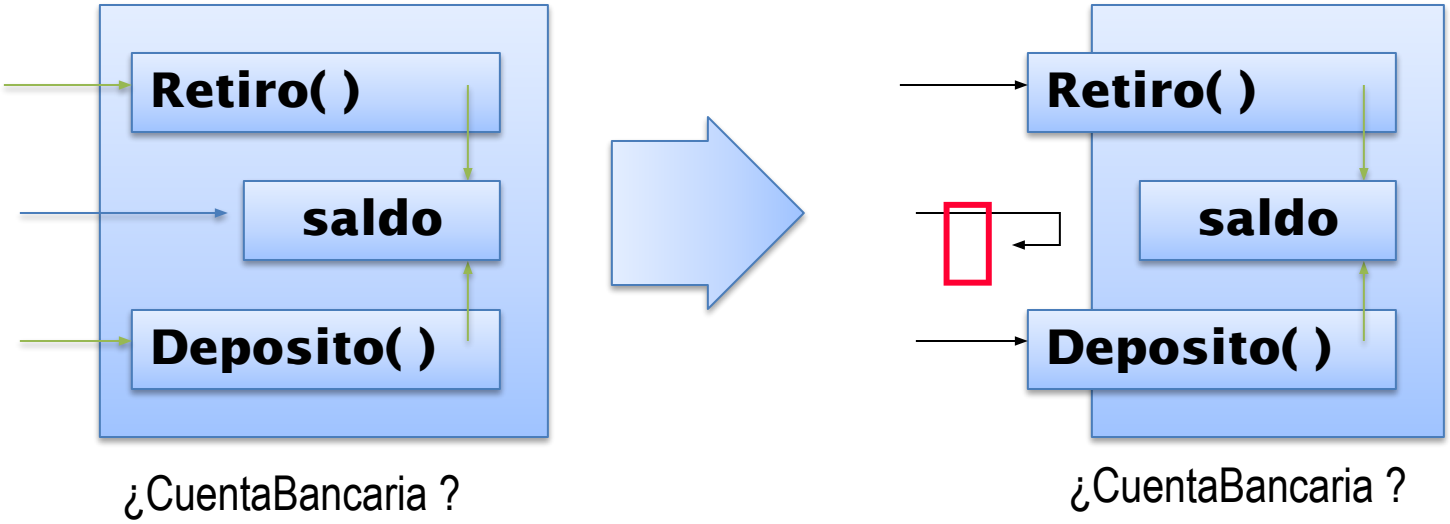
Encapsulación

- Combinación de datos y métodos en una sola *cápsula*
- La frontera de la cápsula crea un espacio interior y otro exterior



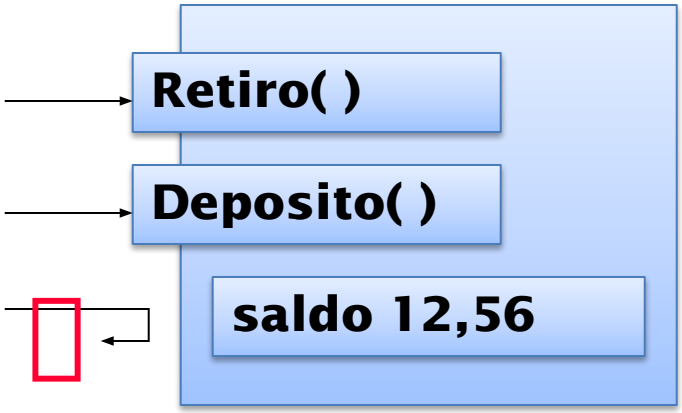
Encapsulación

- Los métodos son ***públicos***, accesibles desde el exterior
- Los datos son ***privados***, accesibles sólo desde el interior

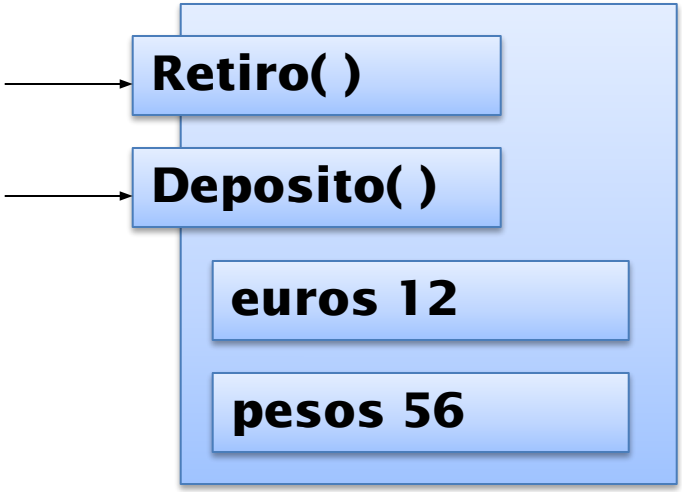


¿Por qué se encapsula?

- Porque permite el control
  - El objeto se usa sólo con los métodos públicos

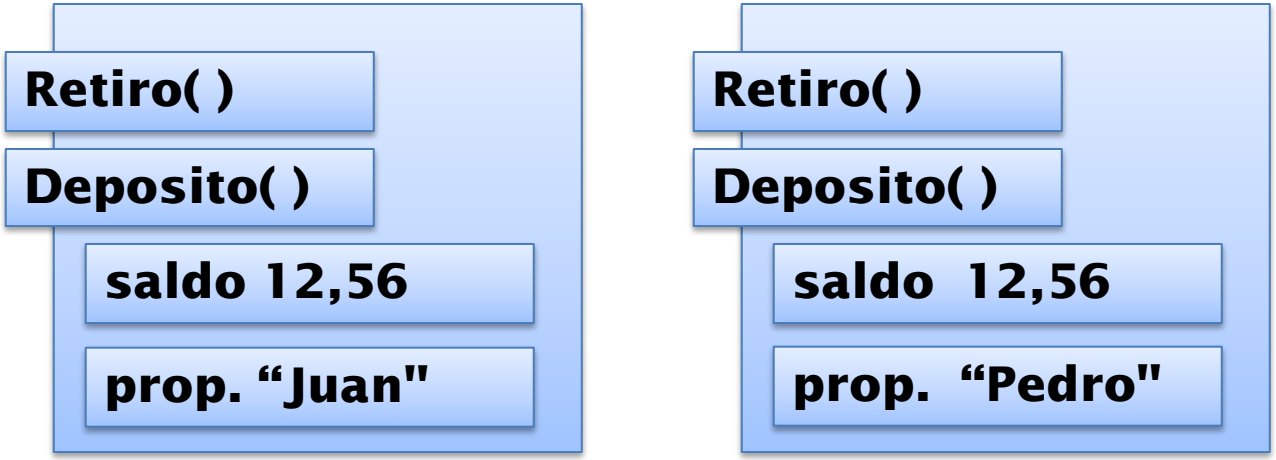


- Porque permite el cambio
  - El uso del objeto no varía si cambia el tipo de los datos privados



Datos de objetos

- Los datos de objetos describen información para objetos *concretos*
  - Por ejemplo, cada cuenta bancaria tiene su propio saldo. Si dos cuentas tienen el mismo saldo, será sólo una coincidencia .



Clases

Sintaxis de definición de clases

- La sintaxis básica para definir una clase es:

```
class <nombreClase>
{
    <miembros>
}
```

Modificadores de acceso

- La encapsulación se consigue añadiendo **modificadores de acceso** en las definiciones de métodos y variables.

- Estos modificadores son instrucciones que se les colocan delante para indicar desde qué códigos puede accederse a ellos.

- Entendiéndose por acceder el hecho de usar su nombre para cualquier cosa que no sea definirlo, como llamarlo si es un método, leer o escribir su valor si es una variable, crear objetos o heredar de él si es una clase, etc.

Tipos de modificadores de acceso

- **public:** Puede ser accedido desde cualquier código.

- **protected:** Desde una clase sólo puede accederse a miembros **protected** de objetos de esa misma clase o de subclases suyas.

- **private:** Sólo puede ser accedido desde el código de la clase a la que pertenece. Es lo considerado por defecto.

Definición de clases simples

- Datos y métodos juntos dentro de una clase
- Los métodos son públicos y los datos privados

```
class CuentaBancaria
{
  private:
  float saldo;
  char nombre[40];
  public:
 void Retiro(float cantidad){
      saldo-=cantidad;
  }
 void Deposito(float cantidad){
      saldo+=cantidad;
  }
  float ObtenerSaldo( ){
      return saldo;
  }

}
```

Campos privados describen un estado inaccesible

Métodos públicos describen un comportamiento accesible

Creación de objetos

```
int main( )
{
  CuentaBancaria micuenta;

  micuenta.Deposito(2000);
  micuenta.Retiro(500);

  cout << micuenta.ObtenerSaldo();

  return 0;
}
```